

エネミー管理(外部ファイル情報からの配置)

複数のエネミーをステージに配置しようとする、指定座標がマジックナンバーになり易く、定数にするにもコード量が多くなり、可読性が低くなります。

敵のパラメータや座標は、外部ファイルに持っておくことが望ましく、BlockActionで行ったCSV読み込みを使用して、エネミーを配置していきます。

今回もExcelを使用して元データを作り、CSVファイルとして出力していきます。

エネミーのデータ

	A	B	C	D	E	F
1	ID	敵種別	HP	初期座標X	初期座標Y	初期座標Z
2	1	0	2	0.0f	100.0f	1500.0f
3	2	0	2	800.0f	100.0f	1500.0f
4	3	0	2	0.0f	100.0f	2000.0f

Excelからcsvファイル出力

名前を付けて保存

EnemyData

CSV (コンマ区切り) (*.csv)

保存

保存場所

Data/Csv/EnemyData.csv

CSVイメージ

1	ID, 敵種別, HP, 初期座標X, 初期座標Y, 初期座標Z↓
2	1, 0, 2, 0.0f, 100.0f, 1500.0f↓
3	2, 0, 2, 800.0f, 100.0f, 1500.0f↓
4	3, 0, 2, 0.0f, 100.0f, 2000.0f↓
5	[EOF]

このcsvファイルを読み込んで、
一旦、この構造体に格納して、Enemyのインスタンスを作成していきます。

```
// エネミーデータ
struct EnemyData
{
    int id;
    EnemyBase::TYPE type;
    int hp;
    VECTOR defaultPos;
};
```

csvの設計と、構造体を合わせておくことで、
csv読み込み時の型の変換であったり、どの列がどの情報なのか、
どの値を代入すれば良いのかなど、わかりやすくプログラミングできます。

EnemyBase.h

public:

```
// 種別
enum class TYPE
{
    RAT
};

// エネミーデータ
struct EnemyData
{
    int id;
    EnemyBase::TYPE type;
    int hp;
    VECTOR defaultPos;
};

// コンストラクタ
EnemyBase(const EnemyBase::EnemyData& data);
```

～ 省略 ～

protected:

```
// 種別  
TYPE type_;
```

```
// HP  
int hp_;
```

EnemyBase.cpp

```
EnemyBase::EnemyBase(const EnemyBase::EnemyData& data)  
:  
    CharactorBase(),  
    type_(data.type),  
    hp_(data.hp)  
{  
    // 初期座標の設定  
    transform_.pos = data.defaultPos;  
}
```

EnemyRat.h

public:

```
// アニメーション種別  
enum class ANIM_TYPE  
{  
    IDLE,  
};
```

```
// コンストラクタ  
EnemyRat(const EnemyBase::EnemyData& data);
```

EnemyRat.cpp

```
EnemyRat::EnemyRat(const EnemyBase::EnemyData& data)
:
    EnemyBase(data)
{
}

void EnemyRat::InitTransform(void)
{
    transform_.scl = VScale(AsoUtility::VECTOR_ONE, SCALE);
    transform_.quaRotLocal = Quaternion::Euler(ROT);
    transform_.pos = { 0.0f, 100.0f, 1500.0f };           ⇒削除
    transform_.Update();
}
```

Application.h

public:

～ 省略 ～

static const std::string PATH_CSV;

Application.h

```
const std::string Application::PATH_IMAGE = "Data/Image/";
const std::string Application::PATH_MODEL = "Data/Model/";
const std::string Application::PATH_EFFECT = "Data/Effect/";
const std::string Application::PATH_CSV = "Data/Csv/";
```

AsoUtility.h

public:

～ 省略 ～

// 文字列の分割

static std::vector<std::string> Split(std::string& line, char delimiter);

AsoUtility.cpp

```
std::vector<std::string> AsoUtility::Split(std::string& line, char delimiter)
{
    std::istringstream stream(line);
    std::string field;
    std::vector<std::string> result;

    while (getline(stream, field, delimiter)) {
        result.push_back(field);
    }

    return result;
}
```

EnemyManager.h

```
#pragma once
#include <vector>
#include "EnemyBase.h"

class EnemyManager
{
public:

    ~ 省略 ~

    // CSVから敵情報の読取を行う
    void LoadCsvData(void);

    // エネミー生成
    EnemyBase* Create(const EnemyBase::EnemyData& data);
```

EnemyManager.cpp

```
#include <string>
#include <fstream>
#include "../Application.h"
#include "../Utility/AsoUtility.h"
#include "EnemyRat.h"
```

```

#include "EnemyManager.h"

void EnemyManager::Init(void)
{

    // エネミーのデータ読み込み
    LoadCsvData();

}

void EnemyManager::LoadCsvData(void)
{

    // ファイルの読込
    std::ifstream ifs = std::ifstream(Application::PATH_CSV + "EnemyData.csv");
    if (!ifs)
    {
        // エラーが発生
        return;
    }

    // ファイルを1行ずつ読み込む
    std::string line;// 1行の文字情報
    std// 1行を1文字の動的配列に分割

    bool isHeader = true;

    while (getline(ifs, line))
    {

        if (isHeader)
        {
            isHeader = false;
            continue;
        }

        // 1行をカンマ区切りで分割
        strSplit = AsoUtility::Split(line, ',');

        EnemyBase* enemy = nullptr;
    }
}

```

```

// 構造体に合わせて読込データを格納
EnemyBase::EnemyData data = EnemyBase::EnemyData();
int idx = 0;
// ID
data.id = stoi(strSplit[idx++]);
// 種別
data.type = static_cast<EnemyBase::TYPE>(stoi(strSplit[idx++]));
// HP
data.hp = stoi(strSplit[idx++]);
// 初期座標
data.defaultPos =
{
    stof(strSplit[idx++]),
    stof(strSplit[idx++]),
    stof(strSplit[idx++])
};

// エネミー生成
Create(data);

}

ifs.close();

}

EnemyBase* EnemyManager::Create(const EnemyBase::EnemyData& data)
{
    EnemyBase* enemy = nullptr;

    ○○○
    ○○○
    ○○○

    return enemy;
}

```

解説

- csvのデータ形式と構造体を合わせておくことで、読み込みプログラム側では、データ順を意識せずに、制作できる

```
data.id = stoi(strSplit[idx++]);
```

```
struct EnemyData
{
    int id;
    EnemyBase::TYPE type;
    int hp;
    VECTOR defaultPos;
};
```

	A	B	C	D	E	F
1	ID	敵種別	HP	初期座標X	初期座標Y	初期座標Z
2	1	0	2	0.0f	100.0f	1500.0f
3	2	0	2	800.0f	100.0f	1500.0f
4	3	0	2	0.0f	100.0f	2000.0f

- stof関数は、string型から、float型へ変換

```
stof(strSplit[idx++]),
```

- EnemyBaseのコンストラクタ時にEnemyData構造体を渡すことで、Enemyのパラメータや初期座標をセットできる

```
EnemyBase(const EnemyBase::EnemyData& data);
```

【要件】

EnemyManagerクラスのCreate関数を完成させること。

- 敵種別に応じた派生クラスのインスタンスを生成すること
- 生成後、Init関数を実行すること
- Init関数実行後、配列に追加すること
- 現在は使用しないが、return で生成したインスタンスを返すこと

【目標】

ステージの下图の位置にネズミが3体描画されていること。

